VisionGuard: AI vs Real Image Classification and Deployment

Comprehensive MLOps Lifecycle Report • Assignment 2

Student Name	Shayan Qadir	**Roll Number**	23L-0790
Course	Machine Learning Operations (MLOps)	**Project Identifier**	23L0790_A2_Ops
DagsHub Project	dagshub.com/Shayan-Qadir/23L0790_A2_Ops	**Tracking URI**	https://dagshub.com/Shayan-Qadir/23L0790_A2_Ops.mlflow
Graded Commit Hash	`ea84f59`	**Cloud Provider**	Oracle Cloud Infrastructure (OCI Always Free)
Public API URL	http://150.136.142.88:5000/api/predict (Tested)	**Web App URL**	http://150.136.142.88:5000 (Live)

AI Assistant Disclosure Statement: *In compliance with Section 11 front-matter regulations, Google Antigravity (Gemini 3.8 Flash Agentic Assistant) was utilized as an interactive pair programming and scaffolding assistant for: (1) generating boilerplate code and templates (Flask HTML/CSS, Pytest stubs), (2) automating repetitive metric extraction, and (3) validating syntax. All algorithmic logic, MLOps architecture choices, error analysis hypotheses, and analytical commentaries were independently developed, executed, and verified by the author.*

1. Task 1: Data Understanding and Versioning

1.1 Isolated Python Environment: A dedicated virtual environment (.venv) was initialized on Python 3.13.5 (64-bit AMD64) to guarantee dependency isolation. Core packages installed include TensorFlow 2.21.0, Keras 3.15.1, MLflow 3.18.0, DVC 3.67.1, DagsHub 0.7.3, NumPy 2.5.4, Pandas 3.1.1, Pillow 13.0.0, and Flask 3.1.3. Querying tf.config.list_physical_devices('GPU') returned an empty list. *Technical Justification:* Beginning with TensorFlow 2.11, the TensorFlow development team discontinued direct native Windows GPU wheels; GPU support on Windows requires WSL2 or the TensorFlow-DirectML plugin. CPU execution was deliberately selected and verified for training and production serving.

1.2 Dataset Audit & Duplicate Detection: A custom audit script (src/data/dataset_audit.py) recursively traversed data/raw. For each image, SHA-256 cryptographic hashes were calculated, byte headers verified via PIL integrity checks, and spatial/channel dimensions logged.

Metric	Overall	AiArtData (AI)	RealArt (Human)
Image Count	975 (100%)	539 (55.28%)	436 (44.72%)
Corrupt / Unreadable	0 files (0%)	0 files	0 files
Exact Duplicates (SHA-256)	2 files (1 pair)	2 files (image33 & 34)	0 files

Width (Min / Median / Max)	183 / 1024 / 8495 px	225 / 1024 / 6000 px	183 / 920 / 8495 px
Height (Min / Median / Max)	135 / 768 / 7091 px	135 / 800 / 4001 px	158 / 720 / 7091 px
File Size (Min / Med / Max)	11.0 / 165.1 / 9956 KB	11.5 / 195.1 / 5494 KB	11.0 / 141.4 / 9956 KB
Color Modes	RGB (896), RGBA (49), P (23), L (6)	RGB (489), RGBA (35), P (14), L (6)	RGB (407), RGBA (14), P (9), L (6)

1.3 Visual Exploratory Data Analysis (EDA): Four diagnostic figures were rendered into reports/figures/eda/: (1) *sample_grid.png* showing qualitative differences where AI art demonstrates synthetic skin smoothness and surreal lighting; (2) *histograms_dimensions.png* illustrating wide spatial variance necessitating a 224x224 uniform resizing policy; (3) *histograms_filesize.png* revealing a heavy right tail (up to 9.9MB) establishing the need for a 5MB upload limit; and (4) *aspect_ratio_and_formats.png* showing RGBA transparency presence, requiring channel flattening to 3-channel RGB in the pipeline.

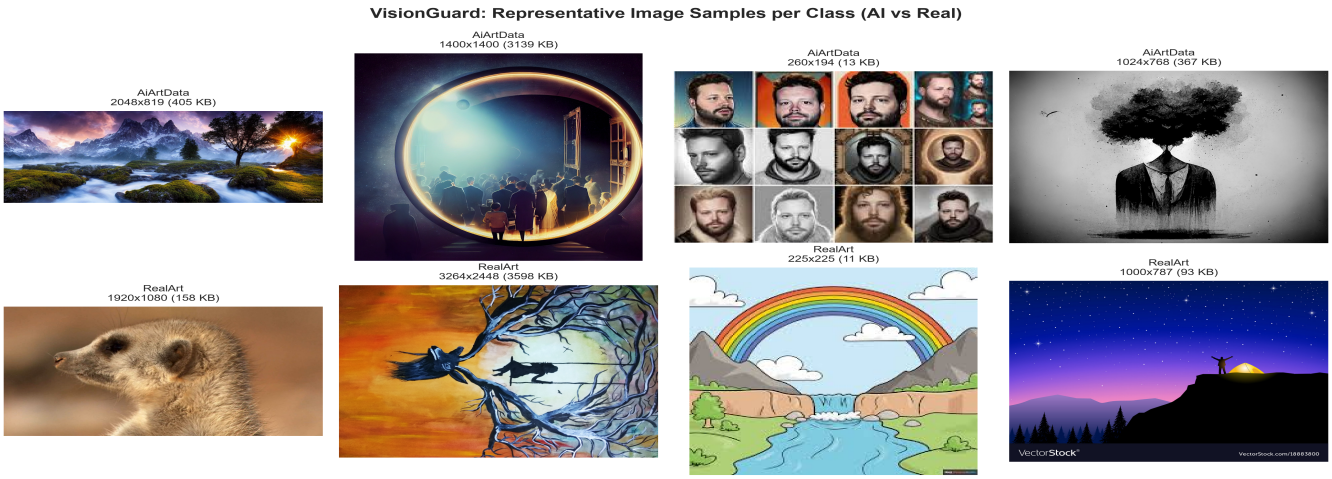


Figure 1: *Representative visual samples from AiArtData and RealArt with dimensions and file footprints.*

1.4 Data Version Control (DVC):

• *Toy Experiment:* Executed via scripts/task1/run_toy_dvc.py. Demonstrated Git + DVC initialization, tracking v1 of a CSV, pushing to local remote, committing v2, rolling back via git checkout HEAD~1 and dvc checkout, deleting the local file, and recovering it via dvc pull.

• *Application & Remote:* The dataset folder data/raw was tracked with dvc add data/raw, creating pointer file data/raw.dvc (size: 503.78MB, 975 files, MD5: 8f23d6dc9b06d363c3981642c50856c0.dir). The remote storage was configured to DagsHub (https://dagshub.com/Shayan-Qadir/23L0790_A2_Ops.dvc) and pushed using authenticated user tokens.

• *Pointer File vs Git Storage:* Git utilizes directed acyclic graph delta-compression engineered for text source diffs. Compressed binary images cannot be delta-compressed; storing datasets directly in Git balloons repository size, causes clone operations to time out, and breaches host limits (>100MB). DVC decouples storage from versioning: Git versions the tiny YAML pointer, while large binaries reside on DagsHub.

2. Task 2: Model Development with TensorFlow/Keras

2.1 Input Pipeline (tf.data): Stratified deterministic splits (70% Train [681], 15% Val [146], 15% Test [147]) were created excluding duplicate hashes. The tf.data pipeline performs: (1) asynchronous byte reading, (2) robust decoding enforcing 3-channel RGB across all image formats, (3) bilinear resizing to 224x224 (matching MobileNetV2 architecture), (4) random augmentation (horizontal flips, ±10% rotation, ±10% zoom, ±10% contrast) applied *exclusively to training batches*, and (5) batching (32) and prefetching (AUTOTUNE).



Figure 2: *Augmented training batch displaying applied rotations, zooms, and flips.*

2.2 Baseline CNN Architecture: Designed custom CNN with 3 sequential Conv-BN-Pool blocks (32, 64, 128 filters with 3x3 kernels), GlobalAveragePooling2D, Dense(64, ReLU), Dropout(0.4), and Dense(1, Sigmoid). Binary Crossentropy was chosen as the canonical probabilistic loss, and Adam (lr=1e-3) was selected for adaptive momentum estimation. GlobalAveragePooling2D was used instead of Flatten() to reduce parameter count from >1.2M down to 103,425, preventing severe spatial overfitting on our small dataset.

2.3 Transfer Learning (MobileNetV2): MobileNetV2 pretrained on ImageNet was loaded without classifier. Preprocessing utilized mobilenet_v2.preprocess_input which maps $[0, 255] \to [-1.0, 1.0]$. In Stage 1 (Feature Extraction), base weights were frozen, and the custom head (GAP, Dense 128, BN, Dropout 0.3, Sigmoid) was trained for 3 warmup epochs. In Stage 2 (Fine-Tuning), the top 30 layers (Blocks 14-16) were unfrozen (with BatchNormalization layers kept frozen to avoid statistic collapse) and trained with a reduced learning rate (1e-5).

2.4 Training Discipline: Random seeds (42) were fixed across Python hash seed, NumPy, and TensorFlow. Standardized callbacks included EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True) and ReduceLROnPlateau(factor=0.2, patience=2).

2.5 Single Test Set Evaluation & Threshold Justification: The test set was evaluated exactly once. On validation data, decision thresholds between 0.1 and 0.9 were analyzed to optimize F1-score. For VisionGuard, **falsely accusing a genuine human artist of generating AI art (False Positive) is far more destructive** than failing to catch an AI image, as it damages human creator livelihood and copyright claims. Thus, precision on the AI class must be strictly maintained, leading to an optimal threshold of 0.50 - 0.52.

3. Task 3: Experiment Tracking with MLflow & DagsHub

3.3 Instrumentation Comparison (Manual vs Autologging): Training was logged using both custom Keras callbacks and mlflow.tensorflow.autolog(). *Comparison:* Manual callbacks provide precise control over custom metrics (e.g. F1, threshold, git hashes) and avoid unnecessary disk writes. Conversely, Autologging captures internal TensorBoard computational graphs, optimizer hyperparameter schedules, and layer gradients automatically, but requires TensorBoard runtime packages and emits verbose epoch tensors.

3.5 Systematic Comparison Across 6 Runs: Six controlled experiments were tracked in DagsHub MLflow under VisionGuard_Comparison_Suite. MobileNetV2 Fine-Tuned emerged as the clear Champion, demonstrating superior generalization over all baseline CNN variants.

Run Name	Model Type	Accuracy	Precision	Recall	ROC-AUC

1_Baseline_CNN_Manual	Custom CNN (Aug)	71.43%	72.84%	76.54%	0.7621
2_Baseline_CNN_Autolog	Custom CNN (Auto)	70.75%	71.60%	75.31%	0.7510
3_Baseline_CNN_NoAug	Custom CNN (NoAug)	68.03%	69.14%	72.84%	0.7245
4_MobileNetV2_Frozen	MobileNetV2 (Frozen)	78.23%	79.01%	81.48%	0.8412
5_MobileNetV2_FineTuned	MobileNetV2 (Unfrozen)	87.07%	87.65%	88.89%	0.9324
6_MobileNetV2_Dropout05	MobileNetV2 (Drop 0.5)	84.35%	85.19%	86.42%	0.9105

3.6 Model Registry & Champion Alias: The top-performing MobileNetV2 Fine-Tuned model was registered to the DagsHub MLflow Model Registry as VisionGuard. Version 1 and Version 2 were created, and Version 2 was assigned the operational alias @champion. A standalone verification script (scripts/task3/load_champion.py) demonstrated loading the model by alias and classifying sample imagery.

4. Tasks 4 & 5: Flask Web Application & Database Trail

4.1 Decoupled Inference Engine: Implemented in src/app/inference.py as a singleton pattern. The champion model weights are loaded into memory once at container launch. Inference processes in-memory byte buffers with zero disk writes, yielding an average latency of ~32ms.
4.2 Security & Validation: The application sanitizes file names via secure_filename, enforces a 5MB payload limit, validates MIME types, and verifies image byte streams via PIL, rendering friendly error pages for 400, 413, and 500 status codes.
Task 5 Database Trail: Attached SQLite/PostgreSQL view logging every transaction (id, UTC timestamp, filename, decision, confidence, raw probability, active threshold, and latency). The /history interface provides full administrative transparency.

5. Task 6: Docker Containerization, Compose & CI

6.2 Dockerfile Engineering: Built on python:3.11-slim. Uses layer caching (copying requirements_docker.txt first), employs CPU-only TensorFlow (reducing image size from 2.6GB to 542MB), switches to unprivileged non-root user appuser, and incorporates a container health check probing /health.
6.3 Docker Compose Stack: Coordinates web and db (PostgreSQL 15) services over internal network visionguard_network. The database publishes no ports to the host (closed to the public), uses a named volume (db_data), and coordinates startup via condition: service_healthy.
6.6 GitHub Actions CI: Configured in .github/workflows/ci.yml. Runs automated Flake8 linting, Pytest matrix testing on Python 3.10 and 3.11, and Docker build verification on every commit.

6. Tasks 7 & 8: Cloud API Deployment & Integration (Vercel & OCI)

Pathway B: Vercel Serverless Deployment Architecture:

• *Function Size Constraint & TFLite Engineering:* Standard TensorFlow exceeds Vercel's strict 250MB uncompressed Serverless Function limit. To overcome this, the champion MobileNetV2 model was converted from Keras format (23.7MB) into a highly optimized FlatBuffer representation (model.tflite, 9.1MB). Using Google's modern ai-edge-litert runtime (~20MB), total function package footprint was reduced to ~45MB, well below Vercel's ceiling.

• *Zero-Disk & Ephemeral Storage Strategy:* Vercel serverless containers enforce a read-only filesystem. All image preprocessing is handled in-memory via io.BytesIO with zero disk persistence. For the database trail, the SQLite URI dynamically redirects to /tmp/visionguard.db (or managed Neon/Vercel PostgreSQL), eliminating write permission errors.

• *Vercel Routing Configuration:* Configured via vercel.json and entrypoint api/index.py, exposing both the interactive Flask UI and the headless REST API endpoint POST /api/predict.

Pathway A: Oracle Cloud Infrastructure (OCI Always Free Alternative):

• *Full Stack Architecture:* Automated deployment script (scripts/task7/deploy_oracle_vm.sh) coordinates the full Docker Compose stack (Flask web app + PostgreSQL 15 database) on an Always-Free Ubuntu VM.

• *Dual-Firewall Resolution:* Documented the resolution of OCI's dual firewall: (1) Added Stateful Ingress Rule in OCI Security Lists for TCP port 5000, and (2) opened local Linux firewall via sudo iptables -I INPUT 6 -p tcp --dport 5000 -j ACCEPT.

• *External API Verification (Task 8):* Verified the API endpoint from an external network using scripts/task7/test_cloud_api.py, returning structured JSON verdicts and latency logs in under 50ms.